

Individual Mini-Project Report Template

Cloud Computing Module — Mini-Project 1

Project Title	Campus Buzz
Module	COMP3041J-Cloud Computing (S_W Eng)
Submission Type	Individual report (upload via Brightspace Quiz)

Important anonymisation note: Do not include your name, student ID, or group ID anywhere in this report. Also remove identifying details from screenshots, logs, links, or file paths before submission.

Abstract

Word limit: 50–80 words. Briefly state your role and the main component(s) you delivered.

In this project, I acted as the Frontend and Workflow Specialist. My main job was delivering the two entry-point containers: the Presentation Service and the Workflow Service. I focused on building a clean UI for event submissions and setting up the backend logic that kicks off our automated review process. Essentially, I made sure that when a user hits "Submit," the data gets where it needs to go.

I. Individual Role and Responsibilities

Word guide: 250–300 words

Purpose: Clearly demonstrate your own technical contribution to the group project.

A. What I was responsible for

Describe the specific technical tasks you were responsible for (e.g., Dockerfiles, docker-compose configuration, serverless functions, cloud service setup, deployment, testing).

<Write here>

As the owner of the system's entry points, my primary technical responsibility was to bridge the user experience with the cloud-native backend.

On the frontend, I developed the Presentation Service using React and Material-UI. This involved building a responsive form that captures the five mandatory fields: title, description, location, date, and organiser name. I implemented real-time validation, for instance, I added a character counter for the description field because the system will flag anything under 40 characters as needing revision.

On the backend, I was responsible for the Workflow Service. This is a crucial middleman service. When a user submits an event, this service catches it, talks to the Data Service to create a "Pending" record, and then immediately pings our serverless event function to start the actual processing. I spent a lot of time making sure this handover was asynchronous, because I didn't want the user's browser hanging while the cloud functions were doing the heavy lifting in the background. Finally, I containerized both services using Docker, ensuring they were ready for integration into the overall workflow.

B. Artefacts I worked on

List the concrete artefacts you worked on (e.g., file names, services, functions, configuration files).
Examples:

- Dockerfile for service <service-name>
- docker-compose.yml networking and environment configuration
- <function-name> serverless function in <file-name>

<Write here>

Here are the specific pieces of the project I owned:

- services/presentation/index.html: I managed the root HTML structure, ensuring that the React app mounted correctly and that all necessary meta tags and external font dependencies, like Material Icons, were properly loaded for a professional look.
- services/presentation/src/App.js: This was the most substantial part of my frontend work. I wrote the logic for the React app, including the form submission, the input validation, and a polling mechanism that periodically checks the database so users can see when their event status changes from "Pending" to "Approved." To notice, I didn't just build a static form, but implemented a dynamic state-management system using React hooks. I spent a lot of time on the UX logic, specifically the character counter for the description field and the data-polling mechanism. Since the backend processing happens asynchronously in the serverless layer, I implemented an setInterval function that pings the Data Service every few seconds. This ensures that a user's submission transitions smoothly from a "Pending" state to its final "Approved" or "Needs Revision" status without the user having to manually refresh the page.
- services/presentation/Dockerfile: I created a multi-stage Docker build to keep our frontend image slim and fast. The first stage handles the Node.js build process, and the second stage uses a lightweight Nginx image to serve the static assets. This significantly reduced our image size, which is a best practice for containerized cloud deployments.
- services/workflow/main.py: This script is the backbone of our work. I used Flask to set up a dedicated endpoint (/api/submit) that acts as a gatekeeper. It performs the initial validation check to make sure no empty payloads reach our database, then coordinates the interaction between the Data Service and the first Serverless function.
- services/workflow/Dockerfile: I configured this environment to be as lean as possible, using a Python-slim base image. I also had to manage the environment variables within this file to ensure the container knew exactly where to find the other services in our Docker network.

C. Integration with the group system

Explain how your work connected to or depended on other group members' work (inputs/outputs, API calls, shared data, configuration dependencies).

Minimum expectation: Describe at least two concrete tasks, each tied to a specific artefact or decision.

<Write here>

My work acted as the front door for the entire project, so I had to stay in close contact with my teammates. To note, our team is split into 3 parts. Member A is me, responsible for the front pages, mainly controls the interaction and workflow, while Member B is responsible for the data and integration and Member C is responsible for logic and automation.

1. A core requirement of this project was the hybrid architecture. I implemented the critical handover point where my Workflow Service (a container) triggers the Submission Event Function (a serverless function) via an asynchronous HTTP POST request. This specific integration point was where our app transitioned from a standard service model to an event-driven one.
2. I worked closely with Member B to define our API spec. My Workflow Service had to be aware of the database schema managed by the Data Service. I ensured that my initial POST requests created the necessary placeholders for category, priority, and note fields, which were essential for Member C's logic to function correctly later in the pipeline.
3. My frontend design was dependent on the output from Member C's Result Update Function. I had to coordinate with them to ensure the "Short Note" and "Final Status" strings coming out of the serverless tier were in a format that my React components could parse and display visually using the color-coded Chips I implemented.
4. I collaborated with the team to ensure my services used the correct internal Docker network aliases. This allowed my services to communicate with the Data Service using hostnames like `http://data-service:5002` instead of hardcoded IP addresses, ensuring the whole system stayed portable and easy to deploy via our Docker Compose setup.

II. Use of Generative AI in My Task

Word guide: 700–800 words

Purpose: Show responsible, critical, and verifiable use of Generative AI (GenAI). This section assesses how you used GenAI, not how much you used it.

A. GenAI Tools Used

About 100 words. State which tool(s) you used and what you used them for (e.g., concept clarification, code scaffolding, debugging, configuration, code review).

<Write here>

Throughout the development of the Campus Buzz system, I utilized ChatGPT (GPT-4o) and Claude 3.5 Sonnet as primary collaborative tools. My objective wasn't to outsource the project but to use GenAI as a high-level consultant for technical scaffolding and troubleshooting. I specifically leaned on these models to interpret complex Docker networking errors and brainstorm the most efficient way to handle the asynchronous transition between our containerized Workflow Service and the serverless execution tier. But the main logic, draft and refinement are all done by me. Using these tools allowed me to focus more on the high-level architecture while speeding up the implementation of standard UI patterns.

B. Concrete Examples of GenAI Use

About 300–350 words. Describe one or two concrete examples. For each example, explain:

- The problem or task you were working on
- What you asked GenAI to help with
- What it suggested
- What you accepted, modified, or rejected

Expected detail: Enough that a peer can understand what changed; do not include full prompts or large code blocks.

<Write here>

Example 1:

While building the event submission form, I ran into a persistent visual bug where the label for the "Description" field would overlap with the border of the TextField when it was focused. I asked GenAI: "How do I stop a multi-line Material-UI label from clashing with the outlined border when the field is active?" The AI initially suggested a purely CSS-based fix using absolute positioning and z-index tweaks. I rejected this approach because it felt like a hack that might break the responsiveness of the Material-UI grid. After I pushed back, the AI suggested using the notched prop or adjusting the InputLabelProps to include a background color. I accepted and modified the second suggestion, but instead of just adding a background, I created a custom SX style object that applied a specific padding and background color only when the label was in its shrunk state. This was a cleaner solution that respected the library's internal logic rather than fighting against it.

Example 2:

Because our system uses a hybrid model where processing happens in the background, the UI needs to show status changes without the user manually refreshing. I asked GenAI: "What is the most reliable way for a React frontend to track a status change in a database that is being updated by an external serverless function?"

The AI suggested two main paths: implementing WebSockets for a persistent connection or setting up a polling mechanism using setInterval. I rejected the WebSocket idea because it added unnecessary complexity to our infrastructure. I accepted the polling strategy but had to heavily modify the code. The AI's first draft used a naive useEffect that triggered a new fetch on every re-render, which would have crashed our Data Service under a loop of requests. I rewrote the logic to include a backoff strategy and ensured a cleanup function was in place to clear the interval when the component unmounts, preventing memory leaks.

C. Verification and Critical Evaluation

About 200–250 words. Explain how you verified the GenAI-supported work. Examples include:

- Running and testing containers or serverless functions
- Inspecting and understanding generated code
- Comparing suggestions with official documentation or examples
- Identifying incorrect assumptions or misconfigurations

Note: You are assessed on the verification process, not just success. Screenshots, logs, or full prompt transcripts are not required.

<Write here>

I maintained a strict "trust but verify" policy with all AI-generated snippets. My primary verification method was documentation comparison. For instance, when GenAI suggested props for the Material-UI Chip components to display our categories, I cross-referenced the suggested syntax with the official MUI v5 documentation. I discovered the AI was occasionally hallucinating props that existed in v4 but had been deprecated, which allowed me to correct the code before even running it.

Furthermore, I performed isolated functional testing on the Workflow Service. Before connecting the frontend, I used Postman to send JSON payloads to my Flask API. This allowed me to verify that the service was correctly creating records in the Data Service and triggering the Submission Event Function independently. By inspecting the database state and the function logs via the Docker console, I could confirm that the logic was sound. This independent verification ensured that when I finally integrated the UI, I knew any bugs would be frontend-specific rather than systemic.

D. Limitations and Failures of GenAI

About 150–200 words. Describe at least one case where GenAI output was incorrect, incomplete, misleading, or unsuitable. Explain:

- How you detected the issue
- What you changed or fixed
- What you learned

Note: Honest discussion of mistakes is expected and will not be penalised.

<Write here>

One significant failure occurred when I asked GenAI to configure the network communication between the Workflow Service and the Data Service within our Docker environment. The AI confidently provided code using `http://localhost:5002/records` as the target URL.

I detected this issue immediately after running `docker-compose up`, as the Workflow container threw a `ConnectionRefused` error every time it tried to save a record. I realized the AI had completely failed to account for the containerized context. In Docker, `localhost` refers to the internal loopback of the individual container, not the host machine or other services. The AI's suggestion was fundamentally unsuitable for a multi-container architecture.

To fix the error, I had to discard the AI's advice and manually configure the environment variables to use the service name defined in our compose file, `http://data-service:5002`. This taught me that while GenAI is excellent at syntax, it often lacks situational awareness regarding infrastructure. I learned that I must always be the one to define the network topology, as AI tends to default to local, non-containerized assumptions.

III. Reflection on Learning

Word guide: 250–300 words

Purpose: Reflect on technical learning and judgement (not personal opinion).

Reflect on:

- Technical learning (e.g., Docker, multi-container applications, serverless execution/testing)
- Working in a group on a cloud-based system (e.g., dividing responsibilities, integration, resolving deployment/configuration conflicts)
- Cloud application design decisions (e.g., containers vs serverless, service communication, configuration/scaling/failure handling)
- Responsible GenAI use (where it helped, where your judgement was essential)

Optional prompt: If you repeated this task, what technical or design decision would you change, and why?

<Write here>

This project provided a deep dive into the practicalities of hybrid cloud architectures, specifically the trade-offs between containerized persistence and serverless logic. My biggest technical takeaway was understanding the execution model's impact on system responsiveness. Initially, I viewed the Workflow Service as just an API gateway, but I realized its true value is in decoupling. By using a container for the workflow and offloading rules-based checking to Serverless Functions, we achieved a system that handles heavy lifting without blocking the user interface. This taught me that cloud design isn't just about where code lives, but how state and control are handed off between services.

Working in a group highlighted the critical importance of a Contract-First approach. Our early conflicts stemmed from minor discrepancies in the JSON payload structure between the frontend and the Data Service. I learned that in a multi-container environment, clear API documentation is more valuable than the code itself. Once we solidified our schema, our integration became seamless. If I were to repeat this task, I would implement a Message Queue like RabbitMQ between the Workflow and the Serverless tier. Currently, our system relies on direct HTTP calls, which creates a point of failure. A queue would provide better fault tolerance and scalability, ensuring no submission is lost if a function instance throttles.

Finally, this project reshaped my view of GenAI. While it was an excellent tool for generating Material-UI boilerplate and Docker configurations, it frequently failed at architectural empathy, which means it didn't understand how our specific services were networked. My judgment was essential in overriding AI suggestions that ignored container isolation. Ultimately, I learned that while AI can write the lines of code, the developer must remain the architect of the system's connectivity and reliability.

Important Notes

- GenAI may be used to support your work, but not to replace understanding, implementation, or verification.
- GenAI use without verification or critical reflection may receive limited credit.
- Vague contribution claims (e.g., "helped with the backend") may receive limited credit.

References

<Add references if applicable>
